CAPSTONE PROJECT: FINAL REPORT

IndieGuard: Indie Game Launch Analytics

Predicting negative-review risk and identifying winning feature combinations on Steam

Course	23CSE452 Business Analytics (Units 1, 2 and 3)
Programme / section	[programme and section]
Team number	Group 1
Guide	Rohini S
Academic year	2026-2027
Submission date	[submission date]
Repository and board	github.com/Shan713/IndieGuard; github.com/users/Shan713/projects/1
Live dashboard	https://indieguard.streamlit.app

Team member	Register number
Thotakura Srihitha	CB.SC.U4CSE23450
Gayas Sheik	CB.SC.U4CSE23118
Shantharam P	CB.SC.U4CSE23146
Pradeep Sasidharan	CB.SC.U4CSE23232
Valikala Tejaswini	CB.SC.U4CSE23752

Declaration and acknowledgement

We declare that this report and the project it describes are our own work as a team. The data were collected by our own scrapers from public Steam sources; no pre-built dataset was used. Prior studies, algorithms and software are cited in the reference list. Our use of generative-AI tools, what each tool supported, the code and content it affected, and how we verified and corrected it are disclosed in full in Section 8.3. Tools assisted the work; the project's research design, data collection runs, analytical decisions and final conclusions were made, run and checked by the team.

We thank our guide, Rohini S, for guidance and feedback during the capstone, and our course faculty for the structure and rubric that shaped the work. [add any further acknowledgements]

Team member	Signature	Date
Thotakura Srihitha		
Gayas Sheik		
Shantharam P		
Pradeep Sasidharan		
Valikala Tejaswini		

Abstract

A small indie studio has little room for a bad launch: Steam refunds a game played under two hours within 14 days, so a negative review written in that window costs both the rating and often the sale. IndieGuard asks what drives negative reviews, which feature combinations go with strong reception, and whether patches help. We collected our own data with throttled, resumable scrapers on 1,861 indie roguelike and roguelite games (2022 to 2025), 1.65 million reviews and 29,918 patch and sale events, cleaned them under 32 logged rules and split them by game. Exploratory analysis (Unit 1), five model families at review and game level with grid-search tuning (Unit 2), and two Unit 3 methods, text mining and association rule mining, were evaluated on held-out games. Reviews written inside the refund window are 21.2% negative against 8.8% later. A text model detects negative reviews with a PR-AUC of 0.84 and ROC-AUC of 0.97, but store metadata predicts a game's success tier only modestly (macro-F1 about 0.42 to 0.44 against 0.22 for guessing). Ninety-five tag combinations pass false-discovery control and about two-thirds of the testable ones hold on unseen games; patches show no clear effect against matched controls. A deployed Streamlit dashboard integrates the results and a launch-risk scorer. All findings are associations in observational data.

Executive summary

The problem. A small studio cannot tell from public information what to fix before launch, which feature combinations to back, or when a patch is worth the effort. A negative review in a game's first two hours is especially costly because the player may still refund the game.

The evidence. We built our own dataset from Steam (1,861 games, 1.65 million reviews) and tested every claim on games the models never saw.

- **The first two hours matter.** 21.2% of reviews written under two hours of play are negative, against 8.8% later; they make up 29% of all negative reviews. For games priced $20 or more the early figure is 31.8%.
- **Complaints change.** Early negative reviews lean to crashes, multiplayer bugs and controls; later ones to boss design, balance and repetition.
- **Store data alone say little.** A model that sees only store information finds a game's success tier about twice as well as guessing, which is useful for comparing launch plans but not a forecast.
- **Reading the review text works.** A text model flags negative reviews far better than store data (ROC-AUC 0.97 against 0.76 on the same reviews). It detects complaints; it does not forecast them.
- **Focused 2D, pixel, retro and procedural combinations** reach the Strong tier about two-thirds of the time (35% overall); no combination predicts a struggling launch.
- **Patches do not lift ratings by themselves.** Against matched games with no patch, the change is +0.27 points (interval -0.82 to +1.36).

Business value and actions. Our hypotheses for a studio, in priority order: (1) treat the first hour as a standalone product and blind-test it; (2) match price to what the first hour delivers; (3) prefer focused scope; (4) complete the store page; (5) patch for specific problems, not to lift sentiment. In the illustrative scenario in Section 6.3, a first-hour fix that brings early negativity to the level of cheap games avoids about 14 negative reviews per 1,000 reviews; this is a hypothetical estimate, not a measured result. The live dashboard (https://indieguard.streamlit.app) lets a studio explore the market, the tag rules, the complaint themes, the patch results and score a planned game.

Table of contents

Declaration and acknowledgement ii

Abstract iii

Executive summary iv

Table of contents v

List of figures vii

List of tables vii

Abbreviations viii

1. Introduction, Business Objectives and Literature Review 1

    1.1 Context and scope 1

    1.2 Related analytics work and research gap 2

2. Dataset Collection, Validation and Preparation 2

    2.1 Provenance, coverage and challenges 2

    2.2 Data quality, privacy and ethics 3

    2.3 Transformations and usable dataset 4

3. Exploratory Data Analysis and Visualization (Unit 1) 4

4. Feature Engineering and Predictive Modelling (Unit 2) 5

5. Advanced Analytics (Unit 3): Two Selected Methods 7

    5.1 Method 1: Text mining 7

    5.2 Method 2: Association rule mining 8

6. Comparative Analysis and Business Impact 9

    6.1 Comparison of the two Unit 3 methods 9

    6.2 Comparison with existing studies 10

    6.3 KPI-based business impact and scenarios 10

7. Interactive Application, Figure Index and Reproducibility 11

    7.1 Mandatory dashboard 11

7.2 Figure index and reproduction map 12
7.3 Code and environment 12
8. End-to-End Validation, Traceability and Contributions 12
8.1 Completion and validity 12
8.2 Project tracking and individual work 13
8.3 AI use and originality declarations 13
9. Conclusion, Recommendations and Roadmap 14
References 16
Appendix A. Extended data dictionary and validation records 17
Appendix B. Tuning, additional figures and supplementary analysis 17
Appendix C. Implementation details and dashboard user guide 18
Appendix D. Tracking-board evidence, contribution records, AI declaration and similarity report . 19

List of figures

Figure 1. End-to-end workflow: how data collection, Units 1 to 3 and the dashboard connect. 2
Figure 2. Negative-review share by playtime at review and by price inside the 2-hour window (Python/Matplotlib; src/analysis, notebook 02_EDA_Visualizations.ipynb). 5
Figure 3. Cross-validation and test scores for all models at review level, game level and with review text (Python/Matplotlib; python -m src.models.compare). 6
Figure 4. Game-level XGBoost confusion matrices on the 299 test games (default and tuned class scales). 6
Figure 5. SHAP importance of the game-level XGBoost model (top features). 6
Figure 6. Confusion matrix of the text detector on the held-out reviews (row-normalised). 8
Figure 7. Complaint themes among negative reviews by playtime (NMF, eight topics). 8
Figure 8. The eight Strong rules with the highest lift: share of Strong games on training and test games (Python/Matplotlib; reports/results/association_rules_strong.csv). 9
Figure 9. Market explorer, default state (1,490 games). 11
Figure 10. Market explorer after selecting the tag Pixel Graphics (550 games; KPIs and charts update). 11
Figure 11. Tag rules page filtered to stable rules with lift at least 1.5 (37 of 95 rules). 11
Figure 12. Scorer explanation for a game predicted Struggling: reasons for and against, matched rules, and single changes that would help. 11
Figure 13. Patch impact by launch timing (change in positive-review share, points; Python/Matplotlib; notebooks/review2/patch_impact.ipynb). 18
Figure 14. Share of Struggling games by price band, release year, languages, DLC, achievements and review volume with 95% intervals (Python/Seaborn; notebooks/02_EDA_Visualizations.ipynb). 18

List of tables

Table 1. Business objectives, KPIs and success criteria. 1
Table 2. Representative sample of the final data (four games, three reviews; full samples in data/sample/). 3
Table 3. Mapping of key final fields to units and methods. 4

Table 4. Model comparison on the shared held-out test games (metadata only; guessing PR-AUC 0.106, macro-F1 0.22). Validation: 5-fold CV by game; test scored once. 5
Table 5. Text evaluation on 120,487 held-out English reviews (guessing PR-AUC 0.115). 7
Table 6. Association-rule configuration comparison (same 1,191 training and 299 test games for every row). 8
Table 7. Comparison of the two selected Unit 3 methods (their metrics are not interchangeable). 9
Table 8. Comparison with existing studies (direct numerical comparison is not valid: different data, targets and protocols). 10
Table 9. Illustrative scenarios for a first-hour fix (hypothetical effect sizes; sensitivity to the share of reviews written inside the window). 10
Table 10. Figure index and reproduction map (figures are embedded once, in Sections 1 to 7 and Appendix B). 12
Table 11. Objective-to-evidence traceability matrix. 13
Table 12. Individual contributions and evidence. 13
Table 13. Generative-AI disclosure. 14
Table 14. Prioritised recommendations. 14
Table 15. Key cleaning flags. 17
Table 16. Grid-search results: default (grid point 0) against tuned settings on the shared folds 17
Table 17. Cards per assignee on the project board (68 cards, all in Completed; a card with two assignees counts for both). 19

Abbreviations

Abbreviation	Meaning
AUC	Area under the curve (ROC-AUC: receiver operating characteristic; PR-AUC: precision-recall)
CV	Cross-validation
EDA	Exploratory data analysis
FDR	False discovery rate
FP-Growth	Frequent-pattern growth (itemset mining)
LDA	Latent Dirichlet allocation
NMF	Non-negative matrix factorization
PCA	Principal component analysis
SHAP	Shapley additive explanations
SVD	Singular value decomposition
TF-IDF	Term frequency, inverse document frequency

1. Introduction, Business Objectives and Literature Review

1.1 Context and scope

Steam lets players refund a game within 14 days if they have played it for less than two hours. A negative review written in that window can therefore cost a small studio both its rating and the sale [4]. Our stakeholders are indie studios and small publishers who must decide what to build, what to fix before launch and when a patch is worth it. We kept the scope narrow: indie roguelike and roguelite games released between 2022 and 2025 with at least 10 reviews (1,861 games). All findings are associations, and revenue and refund figures are out of scope because Steam does not publish them.

Overall objective: help a studio decide which feature combinations to back, what to fix before the refund window closes, and when to patch. We ask five questions: will a review be negative and how does playtime affect this (Q1); which success tier will a game reach (Q2); which tag, price and Early Access combinations go with strong reception (Q3); what do players complain about (Q4); and how do ratings change after a patch (Q5). Table 1 lists the KPIs and Figure 1 shows how the units fit together.

Table 1. Business objectives, KPIs and success criteria.

Business objective	KPI / unit	Baseline / target	Unit / method
O1. Quantify the refund-window effect (Q1)	Negative share inside vs outside the window (percentage points)	Baseline: 10.4% overall. Target: report the gap with a 95% interval	U1
O2. Detect negative reviews (Q1)	PR-AUC on held-out games	Baseline: guessing, 0.115. Target: clearly above baseline, CV spread reported	U2, U3 M1
O3. Predict the game tier (Q2)	Macro-F1 and recall per tier on held-out games	Baseline: 0.22. Target: clearly above baseline	U2
O4. Find tag combinations tied to Strong reception (Q3)	Rules passing FDR control ($q < 0.10$, lift at least 1.2) and holding on test games	Baseline: none. Target: rules with a held-out check	U3 M2
O5. Find complaint themes (Q4)	Topic diversity, coherence, stability and agreement with a manual check	Baseline: LDA topic model. Target: interpretable, stable themes	U3 M1
O6. Measure patch impact (Q5)	Change in positive-review share against matched no-patch controls (points)	Baseline: 0 points. Target: interval around the difference	U1, extra
O7. Deliver an interactive application	Public dashboard with filters that update results	Target: working link, 5 pages, live scenario	App

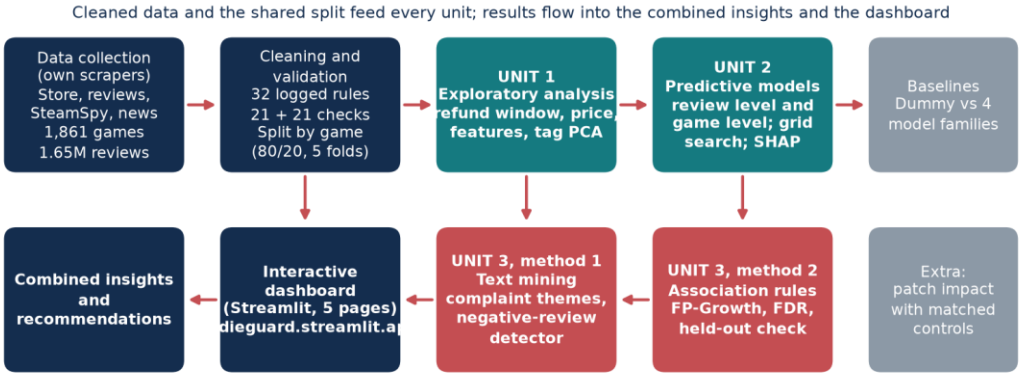


Figure 1. End-to-end workflow: how data collection, Units 1 to 3 and the dashboard connect.

1.2 Related analytics work and research gap

Lin et al. studied about 11 million reviews of 6,224 Steam games and found that negative reviews arrive early and that players criticise design more than bugs [1]. Their work is descriptive and covers the whole platform. De Luisa et al. related price, size, languages, release date and genre to a game's player numbers using Bayesian models [2]; they predict popularity rather than review sentiment and do not use review text. Jorge and Pardo built SteamBR, a corpus of 2.8 million Brazilian-Portuguese reviews, to predict which reviews are helpful [3]. Steam's own review labels and SteamSpy's owner estimates [4, 5] describe games only after the fact. None of these connects early-playtime negativity to what a studio should fix inside the refund window, or links review risk to launch features and patch timing with held-out testing. Text mining and association rules fit this gap: complaints live in free text, and launch decisions are combinations of tags, price and scope.

2. Dataset Collection, Validation and Preparation

2.1 Provenance, coverage and challenges

We collected all data ourselves on 1 and 2 October 2026 from four public Steam sources, using scrapers we wrote: store search (Indie plus five roguelike tags, 2022 to 2025), store details and tags, SteamSpy owner estimates, and the news feed (patch and sale announcements). Reviews were paged with Steam's cursor. No pre-built dataset was used. The endpoints are public and were used for academic analysis at a polite rate; the raw files are frozen with row counts and a hash, and are not redistributed.

Size. The raw data hold 1,861 games (58 variables), 1,652,999 reviews in 31 languages and 29,918 news events; after cleaning, 1,648,387 reviews (34 variables) remain. Observations are at three levels: review, game and announcement. The targets are whether a review is negative (about 10%

are) and a three-level game tier, defined for the 1,490 games with at least 20 reviews: Struggling (30% or more negative), Solid, and Strong (10% or less). Appendix A points to the full data dictionary.

Table 2. Representative sample of the final data (four games, three reviews; full samples in data/sample/).

Game or review	Year / language	Price / playtime	Steam reviews	Negative share	Tier / target
Cross The World	2025	$19.99	23	0%	Strong
Bloody Heaven 2	2024	$4.99	138	15%	Solid
Firework War	2025	$0.00	55	4%	Strong
Godrop	2024	$8.99	313	5%	Strong
Review: The only deck-building game that I...	english	93.1 h			positive
Review: If you're reading comments right n...	english	23.9 h			positive
Review: Best ARPG/Roguelite implementation...	english	38.3 h			positive

Collection challenges. Steam started refusing requests (HTTP 429) after roughly 150 calls, so the scraper waits between requests, backs off when refused and saves its place so a run can resume. SteamSpy was blocked by our network provider and had to be reached through another route. The five largest games were capped at their newest 15,000 reviews; for all other games we collected almost every review Steam reports. No synthetic data were used.

2.2 Data quality, privacy and ethics

Cleaning followed 32 logged rules. We dropped only unusable rows, mainly 4,612 empty reviews (0.28% of the total), corrected 318 wrong launch dates and found no duplicate IDs. Suspicious records (very short or copy-pasted text, reviews written before launch, extreme playtimes, free copies) were flagged rather than deleted, so each analysis could choose its own rule; the counts are in Appendix A. Because only about 10% of reviews are negative, plain accuracy is misleading: always guessing positive already scores about 89%.

Privacy and ethics. Only public data were collected. Reviewer IDs are replaced by a salted hash, and names, avatars and profile links are never stored. This follows the data-minimisation and pseudonymisation principles of India's Digital Personal Data Protection Act, 2023 [6]; all outputs are aggregated or game-level. **Biases:** only about 40% of reviews are in English, games with fewer than 10 reviews are excluded, reviewers choose whether to review, store attributes are an October 2026 snapshot, and the five largest games are capped.

2.3 Transformations and usable dataset

Cleaning is kept separate from model-specific preprocessing. The main transformations were UTC timestamps, a winsorised copy of playtime, corrected launch dates, negation handling in text (for example *not_fun*), a PCA of the tag columns (68 components) and TF-IDF with SVD for review text. Data are split by game, 80/20, with five cross-validation folds inside the training games, so no game appears on both sides. Everything learned from data is fitted on training games only, and an automated script runs 21 leakage and consistency checks. Table 2 shows a sample of the final data and Table 3 maps the main fields to each unit.

Table 3. Mapping of key final fields to units and methods.

Variable	Type / meaning	Origin / preparation	Used in
`voted_up`	bool: recommended or not	Steam review field; target = not voted_up	U1, U2, U3 M1
`author_playtime_at_review`	number (minutes)	Playtime when written; refund window = under 120 min	U1, U2, U3 M1
`review`, `language`	text, category	As written; English subset flagged `text_mining_ok`	U3 M1
`created`	datetime (UTC)	Converted from Unix time; days since launch, patch recency	U1, U2
`price_usd`, `is_free`, `n_dlc`	number, bool	Store details; price tier	U1, U2, U3 M2
`tags`, `genres`, `categories`	text lists	User tags (top 20); 68 PCA components; 137 rule items	U2, U3 M2
`steam_total_reviews/negative`	number	Steam totals; negative share and game tier (outcome, never a feature)	U1, U2 target
`events` (patch, sale)	event table	Steam news, classified; 29,918 rows	U1, extra

3. Exploratory Data Analysis and Visualization (Unit 1)

The exploratory work focused on Q1 and Q3. About 10.6% of the modelled reviews are negative, but the share depends strongly on playtime: reviews written in the first two hours are 21.2% negative, against roughly 8 to 10% later (Figure 2, left). These early reviews make up 29% of all negative reviews, and the pattern holds in 94% of games with enough reviews. Inside the window, negativity also rises with price, from about 11% under $5 to 32% at $20 or more (Figure 2, right). At game level, 14.5% of games are Struggling and 35% Strong.

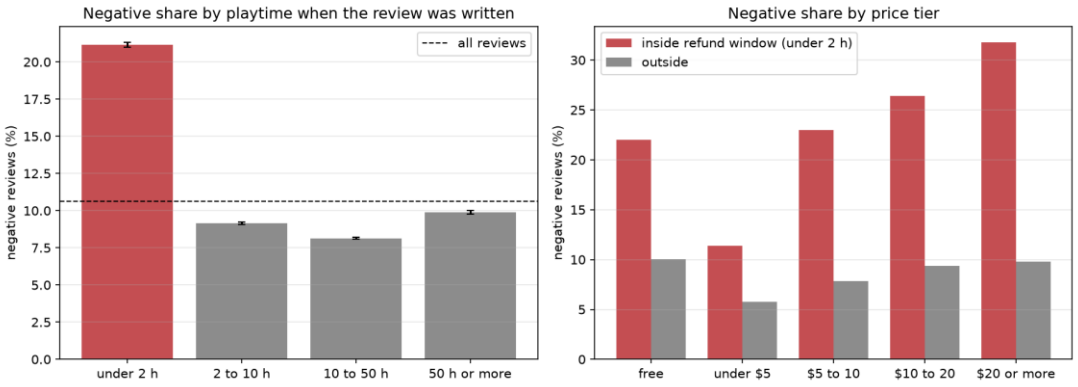


Figure 2. Negative-review share by playtime at review and by price inside the 2-hour window (Python/Matplotlib; src/analysis, notebook 02_EDA_Visualizations.ipynb).

Store signals shift a game's risk by a few points each, and none decides it alone (Figure 14, Appendix B): games with more languages, DLC or achievements show fewer Struggling games. Review counts, owner estimates and refund shares are results of a reception rather than launch signals, so we kept them out of the models. These findings led to three choices: PR-AUC and macro-F1 instead of accuracy, a split by game, and a separate text model for reviews.

4. Feature Engineering and Predictive Modelling (Unit 2)

Targets and predictors. At review level we predict whether a review is negative from playtime, refund-window and launch-week flags, days since launch, recent patches and sales, price and text length. At game level we predict the tier from 83 launch-time features: price, launch year, DLC, achievements, platforms, controller support and 68 tag components. Columns that describe the outcome are never used as features.

Models and protocol. Each task has a Dummy baseline and four models: Logistic Regression, Random Forest, LightGBM and XGBoost [7, 8, 9]. All models share the same split and folds. We tuned each by grid search on the folds, keeping the default setting as one of the candidates (Appendix B, Table 16), and chose thresholds and class scales from out-of-fold predictions; the test set was scored once. **Leakage audit:** preprocessing is fitted on training folds only, outcome columns are excluded, no game is on both sides of the split, and the 21 automated checks pass.

Table 4. Model comparison on the shared held-out test games (metadata only; guessing PR-AUC 0.106, macro-F1 0.22). Validation: 5-fold CV by game; test scored once.

Model	Review CV PR-AUC	Test PR-AUC	Test ROC-AUC	Game CV macro-F1	Test macro-F1 [95% CI]	Recall Struggling
Dummy (prior)	0.107 ± 0.024	0.106	0.500	0.225 ± 0.003	0.220 [0.20, 0.24]	0.00
Logistic Regression	0.208 ± 0.024	0.194	0.663	0.422 ± 0.033	0.423 [0.36, 0.48]	0.39
Random Forest	0.286 ± 0.038	0.280	0.726	0.434 ± 0.037	0.371 [0.31, 0.43]	0.10
LightGBM	0.287 ± 0.037	0.254	0.724	0.445 ± 0.027	0.372 [0.31, 0.43]	0.22
XGBoost	0.292 ± 0.037	0.275	0.729	0.444 ± 0.027	0.437 [0.38, 0.50]	0.41

Results and error analysis. At review level (Table 4, Figure 3) the three tree models are roughly tied at a test PR-AUC of 0.25 to 0.28, about 2.5 times guessing, while Logistic Regression is weaker (0.19). At game level every model beats guessing, but the intervals overlap because the test set has only 299 games. Tuning helped XGBoost (0.437) and Logistic Regression (0.423) on test, while Random Forest and LightGBM fell back to about 0.37 even though their cross-validation scores improved; the search partly fitted the validation folds. We report these numbers as they came out and did not re-tune. Struggling games are the hardest class to find (Figure 4). We selected XGBoost at game level for its test score and Struggling recall, although it is not clearly better than Logistic Regression.

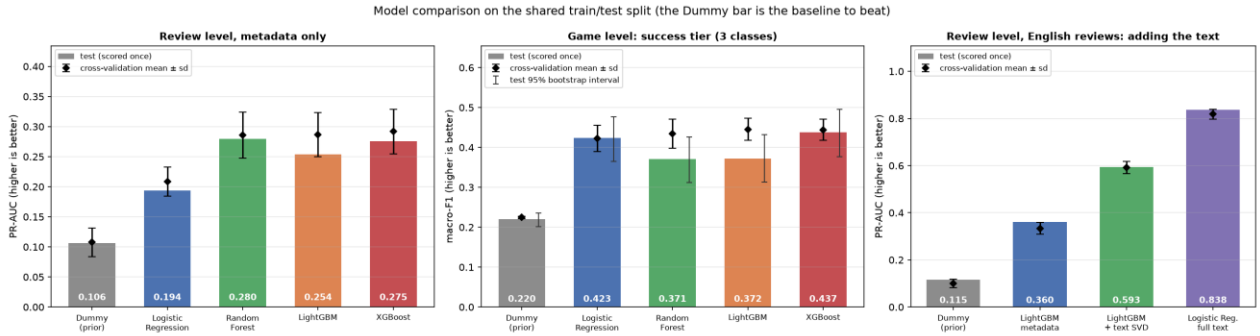


Figure 3. Cross-validation and test scores for all models at review level, game level and with review text (Python/Matplotlib; python -m src.models.compare).

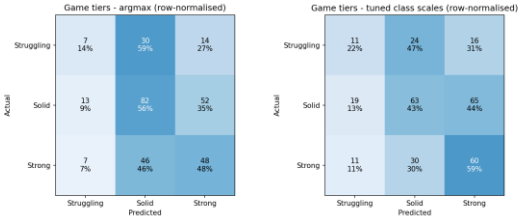


Figure 4. Game-level XGBoost confusion matrices on the 299 test games (default and tuned class scales).

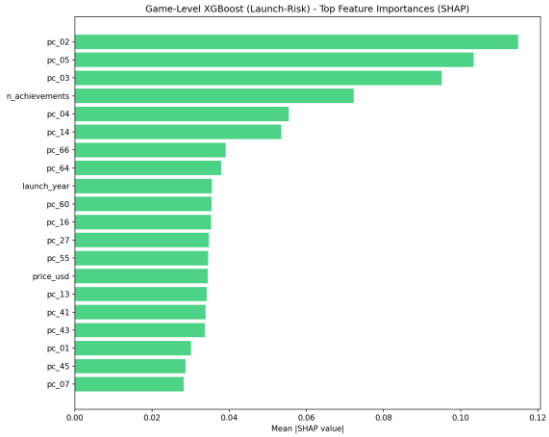


Figure 5. SHAP importance of the game-level XGBoost model (top features).

Interpretation and bias checks. At review level, the model leans on review length, Chinese-language reviews and playtime; at game level, on a few tag components and the number of achievements (Figure 5). These are associations, not causes, and the review-level models use reviewer history collected at scrape time, so they explain reviews rather than predict a launch.

Results are not driven by a few large games: the within-game ROC-AUC (0.75) is slightly above the pooled value. We did not run a formal bias test across languages (Section 8.1).

5. Advanced Analytics (Unit 3): Two Selected Methods

We chose text mining and association rules because the business question needs both what players say and which launch combinations go with strong reception. Time-series analysis was not selected; patch impact is reported as an extra in Section 6.1.

5.1 Method 1: Text mining

Question and data. What do players complain about, and can the review text flag a negative review? We used 557,202 English reviews (436,715 for training, 120,487 for testing). The text was cleaned, negations were joined to the next word, and TF-IDF features were built from training reviews only [10]. **Methods:** (a) a Logistic Regression on TF-IDF plus metadata to detect negative reviews, tuned by grid search with the threshold set on out-of-fold predictions; (b) an NMF topic model [11] with eight topics on 40,000 negative training reviews, compared with LDA [12].

Table 5. Text evaluation on 120,487 held-out English reviews (guessing PR-AUC 0.115).

Approach	CV PR-AUC	Test PR-AUC	Test ROC-AUC	Precision / recall	F1
Dummy (prior)	0.101 ± 0.018	0.115	0.500	0.12 / 1.00	0.21
LightGBM, metadata only	0.334 ± 0.025	0.360	0.758	0.32 / 0.49	0.39
LightGBM, metadata + text components	0.592 ± 0.026	0.593	0.895	0.52 / 0.62	0.56
Logistic Regression, full text + metadata	0.819 ± 0.021	0.838	0.971	0.75 / 0.79	0.77

Evaluation and results. Compressed text features lift PR-AUC from 0.36 to 0.59, and the full TF-IDF model reaches 0.84 (ROC-AUC 0.97). On the test reviews it catches 10,980 negative reviews and misses 2,893, with 3,607 false alarms (Figure 6); accuracy is 94.6% against 88.5% for always guessing positive. NMF scored lower than LDA on UMass coherence but produced more distinct, stable topics, so we used it for interpretability. Early negative reviews lean towards crashes, multiplayer bugs and controls, later ones towards boss design and balance (Figure 7); a word such as *boring* appears in 9.1% of negative reviews against 0.8% of positive ones.

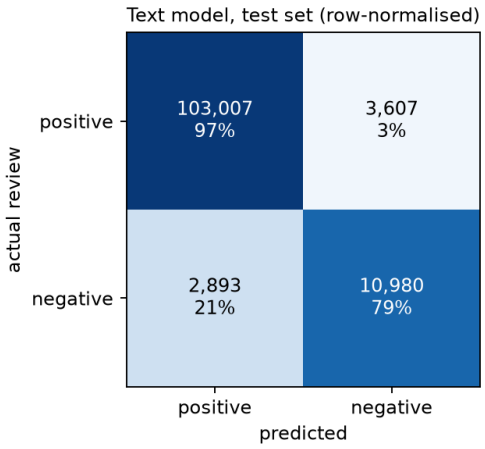


Figure 6. Confusion matrix of the text detector on the held-out reviews (row-normalised).

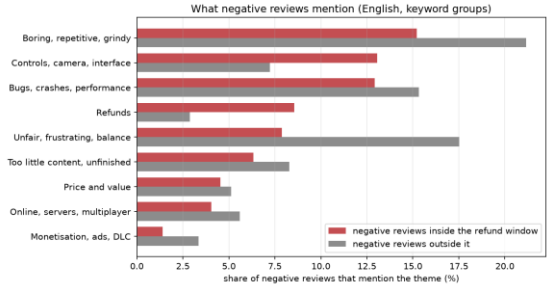


Figure 7. Complaint themes among negative reviews by playtime (NMF, eight topics).

Validation and limitations. In a blind check of 200 reviews, labelled by an AI assistant without seeing the model's topic, the labels matched the model on 35.5% (kappa 0.30). The Early Access, repetition and crash topics were recovered well; the boss and balance topics were not, so we treat those as exploratory. An earlier figure of 96% was withdrawn because those labels were not independent; a human re-label would make the check stronger. The detector identifies complaints in existing reviews; it does not forecast them.

5.2 Method 2: Association rule mining

Question and data. Which tag, price and Early Access combinations go with the Strong tier? Each of the 1,191 training games became a transaction of 137 possible items: its user tags (without opinion tags such as *Great Soundtrack*, which would be circular), a price band and Early Access. **Method:** FP-Growth [13] via mlxtend, keeping itemsets of up to three items seen in at least 25 games, rules with a lift of at least 1.2, and a false-discovery control (Benjamini-Hochberg, q below 0.10) [14]. Each surviving rule was then checked once on the 299 test games.

Table 6. Association-rule configuration comparison (same 1,191 training and 299 test games for every row).

Configuration	Strong rules (FDR)	Mean confidence / lift	Hold on test games
FP-Growth, support 25, up to 3 items (chosen)	95	0.59 / 1.68	68% of 66
Apriori, same thresholds	95	0.59 / 1.68	68% of 66
FP-Growth, support 15 (looser)	107	0.65 / 1.85	66% of 56
FP-Growth, support 40 (stricter)	69	0.57 / 1.60	69% of 59
FP-Growth, pairs only	41	0.56 / 1.59	83% of 30
FP-Growth, keeping after-play tags	131	0.60 / 1.69	69% of 88

Results. Of 4,715 candidate rules, 95 pass for the Strong tier and none for Struggling, so **no tag combination predicts a struggling launch**. About two-thirds of the rules that could be tested (45 of 66) still hold on unseen games. The strongest, Pixel Graphics with Retro and Shooter, covers 25 training games, 80% of them Strong (lift 2.26), and 3 of 4 test games (Figure 8). FP-Growth and Apriori return the same rules; a lower support finds more rules that hold less often (Table 6).

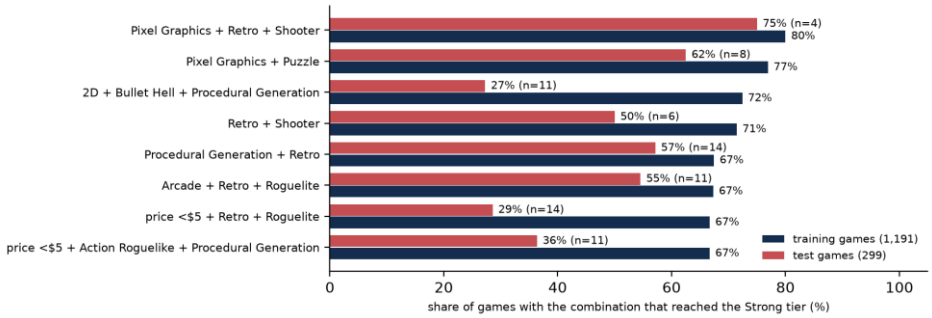


Figure 8. The eight Strong rules with the highest lift: share of Strong games on training and test games (Python/Matplotlib; reports/results/association_rules_strong.csv).

Limitations. Each rule rests on only 30 to 60 training games and a handful of test games, and rules involving a price under $5 are often unstable. They are hypotheses to test, not recipes.

6. Comparative Analysis and Business Impact

6.1 Comparison of the two Unit 3 methods

Table 7. Comparison of the two selected Unit 3 methods (their metrics are not interchangeable).

Criterion	Method 1: text mining	Method 2: association rules
Question	What do players complain about; can text flag a negative review?	Which launch combinations go with Strong reception?
Input and output	557,202 English reviews; detector scores and eight themes	1,191 training games as itemsets; 95 Strong rules
Validation	Held-out reviews from unseen games; PR-AUC, ROC-AUC; topic diversity and stability; blind check	FDR control; each rule checked once on 299 test games; stability
Strengths	Strong, well-validated detector (ROC-AUC 0.97); themes explain complaints	Transparent rules a studio can read; launch-time information only
Limits	Detects, does not forecast; weak topic agreement (35.5%); English only	Small supports; no Struggling rule; associations only

The two methods complement each other, and their metrics are not comparable. Text mining tells a studio *what players object to and when*; the rules tell it *which launch choices tend to go with good reception*. The findings line up across units: complaints cluster in the first two hours and at higher prices (Unit 1; crashes and controls in the text), while store data and tags only partly separate good launches from weak ones (Unit 2 and the rules). As an extra analysis, positive reviews fall by about 2 points after a patch, but matched games without a patch fall by almost the same amount

(difference +0.27 points, interval -0.82 to +1.36), so the drop reflects launch timing and regression to the mean rather than the patch (Appendix B, Figure 13).

6.2 Comparison with existing studies

Table 8. Comparison with existing studies (direct numerical comparison is not valid: different data, targets and protocols).

Criterion	Lin et al. [1]	De Luisa et al. [2]	Current project
Data and setting	About 11M reviews of 6,224 Steam games (2016 snapshot), whole platform	Steam games, popularity as player numbers; price, size, languages, release date, genre	1,861 indie roguelikes, 1.65M reviews collected whole in Oct 2026
Methods and validation	Empirical, descriptive analysis; no model scored on unseen games	Bayesian hierarchical models; coefficient interpretation	Models scored once on held-out games; baselines; FDR-controlled rules; topic model
Comparable findings / limits	Negative reviews come early (we: 21.2% inside 2 h); design criticised more than bugs (we: crashes early, balance later). Not numerically comparable	Price, languages and release date matter for popularity; our game-tier macro-F1 is modest (0.42 to 0.44). Different target, so no numeric comparison	Distinct: refund window, patch timing, text and tags in one niche. Remaining gaps: causal claims, revenue, other genres

A direct numerical comparison is not valid because the datasets, targets and protocols differ. What this project adds is a whole niche collected from scratch, the refund-window view, held-out testing that reports negative results, and an application a studio can use. We do not claim to outperform these studies.

6.3 KPI-based business impact and scenarios

We turned the refund-window finding into an illustrative scenario for one game. **Measured:** about 14% of reviews are written in the first two hours, 21.2% of them negative, against 11.4% for the cheapest games. **Assumed:** a game with 1,000 reviews, a first-hour fix (blind testing, crash and control fixes) costing about 40 person-hours, and an early-negativity drop of 0, 5 or 9.8 points (the last matches the cheapest tier). Sales and refunds are not public, so the benefit is expressed as negative reviews avoided per hour of effort. These are estimates, not measured outcomes.

Table 9. Illustrative scenarios for a first-hour fix (hypothetical effect sizes; sensitivity to the share of reviews written inside the window).

Scenario	Change in early negativity	Avoided negatives per 1,000 reviews (window share 10% / 14% measured / 20%)	Overall negative share (14%)	Avoided per 100 effort-hours
Worst	-0.0 points	0.0 / 0.0 / 0.0	10.4% to 10.4%	0.0
Base	-5.0 points	5.0 / 7.0 / 10.0	10.4% to 9.7%	17.5
Best	-9.8 points	9.8 / 13.7 / 19.6	10.4% to 9.0%	34.3

The gain is modest, about 7 fewer negative reviews per 1,000 in the base case, but those are the reviews most likely to come with a refund, and the estimate depends on how much a fix achieves and how many reviews come early. For patches, we found no general effect, so patching only to lift

sentiment is not supported by our data; patching specific crashes and controls still makes sense. Section 9 ranks the recommendations.

7. Interactive Application, Figure Index and Reproducibility

7.1 Mandatory dashboard

Access. The dashboard is public at https://indieguard.streamlit.app; the free host sleeps when idle, so it should be opened a few minutes before a demo. It can also be run locally with `streamlit run src/dashboard/app.py`. **Design.** It is built with Streamlit, pandas and Altair, reads small tables exported from the analysis and one saved XGBoost model. Unit 1 summaries and the Unit 3 rule, topic and patch results are precomputed; filters, charts and the launch-risk scorer are computed live. It has five pages: market explorer, tag rules and complaints, patch impact, launch-risk scorer and an about page.

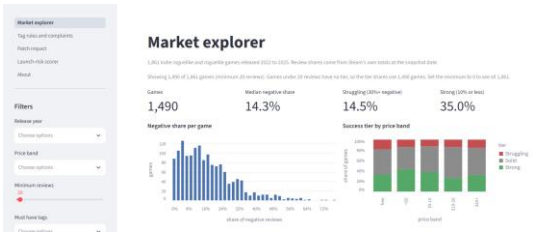


Figure 9. Market explorer, default state (1,490 games).

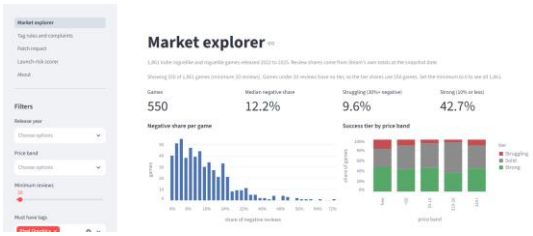


Figure 10. Market explorer after selecting the tag Pixel Graphics (550 games; KPIs and charts update).



Figure 11. Tag rules page filtered to stable rules with lift at least 1.5 (37 of 95 rules).

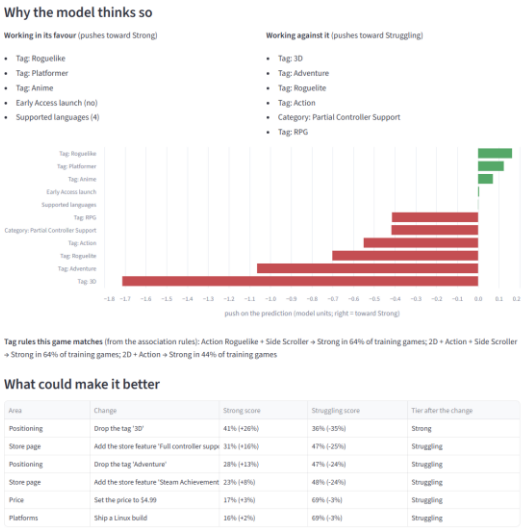


Figure 12. Scorer explanation for a game predicted Struggling: reasons for and against, matched rules, and single changes that would help.

Review 2 feedback. The panel asked that the scorer explain its output. The scorer now shows why a game looks strong or weak, using the model's per-feature contributions (each tag is scored by removing it and re-scoring), lists the tag rules the game matches, and suggests single launch-time changes, each re-scored by the same model (Figure 12). **Tests and limits.** The saved feature pipeline reproduces the stored features for all 1,861 games, and every page was opened in the deployed app and in an automated browser run. The scores are class-scaled, not probabilities (about 0.43 macro-F1), the explanations are associations, and the data are an October 2026 snapshot.

7.2 Figure index and reproduction map

Table 10. Figure index and reproduction map (figures are embedded once, in Sections 1 to 7 and Appendix B).

Figure	Title / tool	Section / purpose	Source notebook / file
Fig. 1	End-to-end workflow, Matplotlib	1.1: how units connect	`build_capstone_report.py`
Fig. 2	Refund window and price, Matplotlib	3: core Unit 1 finding	`figures/findings/refund_window.png`; notebook 02
Fig. 3	Model comparison, Matplotlib	4: models vs baselines	`python -m src.models.compare`
Fig. 4, 5	Game confusion; SHAP bars	4: errors, interpretation	`figures/models/game_xgb_*.png`
Fig. 6, 7	Text confusion; complaint themes	5.1: detector and themes	`review_text_detail.py`; notebook 03
Fig. 8	Top rules train vs test, Matplotlib	5.2: held-out check	`association_rules_strong.csv`
Fig. 9 to 12	Dashboard screenshots	7.1: interactive states	`figures/dashboard/`
Fig. 13, 14	Patch impact; risk by game features	App. B: 6.1 extra; 3 store signals	`patch_impact.ipynb`; `figures/eda/10_*.png`

7.3 Code and environment

Repository: https://github.com/Shan713/IndieGuard (public). The README describes the folder layout (`src`, `notebooks`, `data`, `figures`, `reports`, `docs`, `models`, `scripts`) and how to install the pinned requirements (Python 3.12). One script rebuilds the features and runs the checks, `python -m src.models.compare` regenerates the model tables, a grid-search script reruns any tuning search, and a build script refreshes the dashboard tables. No secrets are stored; the raw scrape and the hashing salt are left out, and the collection scripts can regenerate the data from Steam's public endpoints within Steam's terms.

8. End-to-End Validation, Traceability and Contributions

8.1 Completion and validity

Everything in the declared scope is implemented and can be shown: data collection and cleaning, EDA, five model families at review and game level with tuning, two Unit 3 methods, the deployed dashboard, this report and the presentation. **Threats to validity.** *Internal:* tuning on 1,191 games overfit the folds for two models, and the topic check had a single AI annotator. *External:* one niche, one snapshot, about 40% English reviews and no revenue data. *Construct:* the review models use

reviewer history collected at scrape time, and the game tier is a rule on the negative share. *Reproducibility:* game-level scores move by about 0.01 between runs. We did not test for language bias or analyse discounts.

Table 11. Objective-to-evidence traceability matrix.

Objective / KPI	Analysis / method	Result / metric	Figure / artifact	Recommendation
O1 Refund window	Section 3, EDA	21.2% vs 8.8% negative; 29% of negatives	Fig. 2	R1, R2
O2 Detect negative reviews	Sections 4, 5.1	PR-AUC 0.838; ROC-AUC 0.971 (guessing 0.115)	Fig. 3, 6; Table 5	R1
O3 Predict game tier	Section 4	Macro-F1 0.42 to 0.44 vs 0.22	Fig. 3, 4; Table 4	R4
O4 Tag combinations	Section 5.2	95 Strong rules; 68% of 66 testable hold; none predict Struggling	Fig. 8; Table 6	R3
O5 Complaint themes	Section 5.1	NMF diversity 90.0%; blind check 35.5%	Fig. 7	R1, R5
O6 Patch impact	Section 6.1	+0.27 points [-0.82, +1.36] vs controls	Fig. 13	R5
O7 Application	Section 7	Public dashboard, 5 pages, filters and scorer	Fig. 9 to 12	R1 to R5 (exploration)

8.2 Project tracking and individual work

Work was tracked on a GitHub Projects board (https://github.com/users/Shan713/projects/1, private, access on request) with the stages Backlog, To Do, In Progress, Review/Testing and Completed; each card has an owner, a due date and evidence. Commits and pull requests are public in the repository. Individual marks may vary with participation and the explanation of assigned work.

Table 12. Individual contributions and evidence.

Team member / register no.	Assigned work	Evidence link / artifact
Thotakura Srihitha / CB.SC.U4CSE23450	Exploratory analysis (29 figures, association tests, tag combinations); dashboard wireframe and downloads; grid-search tuning of the game-level models and review Logistic Regression (shared with Shantharam P)	PRs #1, #101, #102; notebook 02; cards #20, #21, #37, #40, #41, #86 to #89, #91, #93
Gayas Sheik / CB.SC.U4CSE23118	Feature engineering and tag PCA; patch impact with matched controls; launch-risk scorer; reproducibility	PRs #2, #49, #50, #69, #71, #80; cards #4, #18, #19, #38, #39, #42
Shantharam P / CB.SC.U4CSE23146	Data collection and cleaning; split and targets; model comparison; grid-search tuning (game-level and review Logistic Regression shared with Thotakura Srihitha); dashboard and deployment; final report; board	`src/collect`, `src/clean`, `src/models`; PRs #56, #57, #95 to #97, #102; cards #5 to #17, #23 to #27, #43, #45, #83 to #94
Pradeep Sasidharan / CB.SC.U4CSE23232	XGBoost review- and game-level models with SHAP; association rule mining	PRs #55, #70; cards #34, #35, #36
Valikala Tejaswini / CB.SC.U4CSE23752	Text mining and topic modelling; combined business insights; blind topic check	PRs #66, #81, #99; cards #31, #32, #33, #44

8.3 AI use and originality declarations

We list every generative-AI tool we used. The tools assisted the team; they did not replace it. The research question, scope, data collection runs, analytical choices and conclusions were decided by team members, and each member wrote and checked their own modules. By signing the declaration page, each member confirms that this list is complete for their own work.

Table 13. Generative-AI disclosure.

Tool	Tasks supported	Content / code affected	Human verification and corrections
ChatGPT (OpenAI)	Ideation: brainstorming project topics, research questions and feature ideas in the planning stage.	Early ideas and the shortlist of topics and questions. No data, code or reported results.	The team discussed the ideas, checked feasibility against the available public data, and wrote and agreed the problem statement and KPIs. Nothing from the tool is reported as a result.
Antigravity (AI coding assistant)	Minor coding assistance: small code completions and fixes in project scripts.	Small, scattered completions and fixes; no complete module or analysis.	Each change was read, run and tested by the team member who made it; outputs were checked against expected values and the automated checks (Section 8.1).
Claude (Anthropic): Claude chat and Claude Code	Building the Review 2 presentation deck. Development assistance under the team's direction: writing and debugging scripts for data collection and cleaning, modelling and tuning, analysis, the dashboard and report generation; running scripts and notebooks. Drafting documentation and the first draft of this report. Producing the labels of the blind topic check (AI-generated, and reported as such).	Scripts for data collection and cleaning, modelling and tuning, analysis, dashboard and report generation; documentation; slides; this report. Modules by other members (features and PCA, patch impact, XGBoost with SHAP, association rules, text mining, EDA) were written by those members.	Automated checks (21 feature and 21 Review 2 checks); cleaning counts reconciled (1,652,999 to 1,648,387); every reported number generated from `reports/results` by scripts. The team ran the long jobs on its own machines, reviewed pull requests before merging, and checked slides and figures against the underlying CSVs. Errors found and corrected: an empty-vocabulary bug in text processing, an overstated 96% topic-agreement figure (withdrawn; the honest figure is 35.5%), a chart axis bug, an app crash on an empty tag filter, and tuned scores that fell on test (reported, not hidden).

What AI did not do: it did not collect data on its own, create or alter records, or decide the conclusions; every figure is produced by code from the project data. **Originality declaration.** Apart from the cited sources, the ideas, analysis and writing are the team's own, and borrowed ideas, figures and code are cited. **Turnitin similarity:** [__%] against the 15% limit (report in Appendix D).

9. Conclusion, Recommendations and Roadmap

The objectives were met, as the traceability matrix shows (Table 11): we quantified the refund-window effect, built a text detector and complaint themes, predicted game tiers modestly, found and tested tag-combination rules, measured patch impact against controls, and built a public dashboard. The weaker results (modest game-tier prediction, no rule for struggling games, no patch effect) are findings in their own right.

Table 14. Prioritised recommendations.

Recommendation (hypothesis)	KPI effect	Stakeholder	Priority: impact and feasibility
R1. Treat the first hour as a standalone product: blind-test the first 30 minutes, check controls and camera, enable crash reporting on launch day	Early negative share (21.2%)	Studio / QA lead	High impact, high feasibility

Recommendation (hypothesis)	KPI effect	Stakeholder	Priority: impact and feasibility
R2. Match price to what the first hour delivers; above about $15 only if testers confirm depth	Early negativity by price band (11.4% to 31.8%)	Studio / publisher	Medium impact, high feasibility (test it)
R3. Prefer focused 2D, pixel, retro, procedural scope over multiplayer or 3D	Share of Strong games (about 2/3 vs 35%)	Design lead	Medium impact, medium feasibility
R4. Complete the store page (achievements, controller support)	Share of Struggling games (12.5% vs 19.6%)	Marketing / producer	Low to medium impact, high feasibility
R5. Patch for specific problems (crashes, controls, balance), not to lift sentiment	Change in positive share (+0.27 points vs controls)	Engineering	Medium impact, high feasibility

Roadmap. *Now (first month):* apply R1 to the next build and track the early negative share; use the scorer's suggestions on the planned scope; test R2 with a small price experiment. *Next (one to six months):* refresh the data, re-label a sample of reviews by hand, and re-check R3 and R4 on new releases. *Later:* calibrate the scorer and add discount analysis and other genres. Success means the early negative share moves towards the cheap-tier level and about two-thirds of the rules keep holding on new games. Not done in this project: discount impact, causal designs, revenue analysis, a language bias test and a human-labelled topic check.

References

[1] Dayi Lin, Cor-Paul Bezemer, Ying Zou, and Ahmed E. Hassan. 2019. An empirical study of game reviews on the Steam platform. *Empirical Software Engineering* 24, 1 (2019), 170-207. https://doi.org/10.1007/s10664-018-9627-4

[2] Andraž De Luisa, Jan Hartman, David Nabergoj, Samo Pahor, Marko Rus, Bozhidar Stevanoski, Jure Demšar, and Erik Štrumbelj. 2021. Predicting the popularity of games on Steam. *Elektrotehniški vestnik* 88, 4 (2021), 151-162. arXiv:2110.02896.

[3] Gabriel A. Z. Jorge and Thiago A. S. Pardo. 2023. SteamBR: a dataset for game reviews and evaluation of a state-of-the-art method for helpfulness prediction. In *Proceedings of the Brazilian Workshop on Social Network Analysis and Mining (BraSNAM)*.

[4] Valve Corporation. Steam refund policy and user review summary labels. Steam Support and Steamworks documentation. https://store.steampowered.com (accessed October 2026).

[5] SteamSpy. Owner estimates and game statistics. https://steamspy.com (accessed October 2026).

[6] Government of India. 2023. The Digital Personal Data Protection Act, 2023 (No. 22 of 2023).

[7] Fabian Pedregosa et al. 2011. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825-2830.

[8] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*.

[9] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: a scalable tree boosting system. In *Proceedings of KDD '16*. ACM, 785-794. https://doi.org/10.1145/2939672.2939785

[10] Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management* 24, 5 (1988), 513-523.

[11] Daniel D. Lee and H. Sebastian Seung. 1999. Learning the parts of objects by non-negative matrix factorization. *Nature* 401 (1999), 788-791.

[12] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet allocation. *Journal of Machine Learning Research* 3 (2003), 993-1022.

[13] Jiawei Han, Jian Pei, and Yiwen Yin. 2000. Mining frequent patterns without candidate generation. In *Proceedings of SIGMOD '00*. ACM, 1-12. https://doi.org/10.1145/335191.335372

[14] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B* 57, 1 (1995), 289-300.

[15] Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*.

[16] Sebastian Raschka. 2018. MLxtend: providing machine learning and data science utilities and extensions to Python's scientific computing stack. *Journal of Open Source Software* 3, 24 (2018), 638.

[17] Streamlit Inc. Streamlit: the fastest way to build data apps. https://streamlit.io (version 1.66, accessed October 2026).

In-text citation numbers follow ACM numeric style.

Appendix A. Extended data dictionary and validation records

The complete variable-level data dictionary (all 34 review columns, 58 game columns and the event table) is in `docs/data_documentation.md`, section 5, and the feature catalogue is `docs/feature_engineering/feature_catalog.csv`. Validation records are in `docs/cleaning_log.md`, `docs/validation_summary.md` and `data/processed/cleaning_log.json`. Key flags added by cleaning:

Table 15. Key cleaning flags.

Field	Type	Meaning and count
`playtime_outlier`	bool	Playtime at review above the 99.5% quantile (363 h); 8,242 reviews
`is_prerelease`	bool	Written before the game's launch; 30,018 reviews
`low_content`	bool	Under 3 letters or 2 words; 208,724 reviews
`is_ascii_art`, `is_copypasta`	bool	ASCII art (4,951); same 50+ character text 10+ times (2,285)
`text_mining_ok`	bool	English and none of the three flags: the text-mining subset
`received_for_free`	bool	Reviewer received the game for free; 34,394 reviews
`launch_date` vs `release_date`	datetime	Original launch vs 1.0 date; differ for 318 games
`reviews_capped`	bool	Newest ~15,000 reviews only (5 games)

Appendix B. Tuning, additional figures and supplementary analysis

Table 16. Grid-search results: default (grid point 0) against tuned settings on the shared folds.

Search	Metric	Grid points	Default CV	Tuned CV
Review level, Logistic Regression	PR-AUC	11	0.208	0.208
Review level, Random Forest	PR-AUC	13	0.278	0.281
Review level, LightGBM	PR-AUC	25	0.281	0.287
Review level, XGBoost	PR-AUC	37	0.285	0.292
Game level, Logistic Regression	macro-F1	12	0.418	0.422
Game level, Random Forest	macro-F1	60	0.354	0.434
Game level, LightGBM	macro-F1	55	0.417	0.445
Game level, XGBoost	macro-F1	97	0.409	0.444
Review text, Logistic Regression	PR-AUC	30	0.792	0.794
Review text, LightGBM metadata	PR-AUC	25	0.332	0.334
Review text, LightGBM metadata + text	PR-AUC	25	0.574	0.592

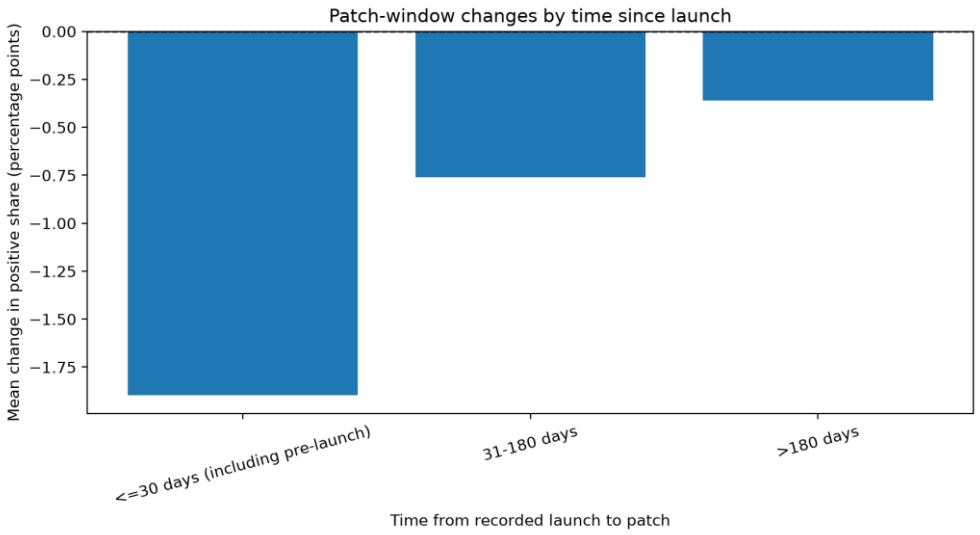


Figure 13. Patch impact by launch timing (change in positive-review share, points; Python/Matplotlib; notebooks/review2/patch_impact.ipynb).

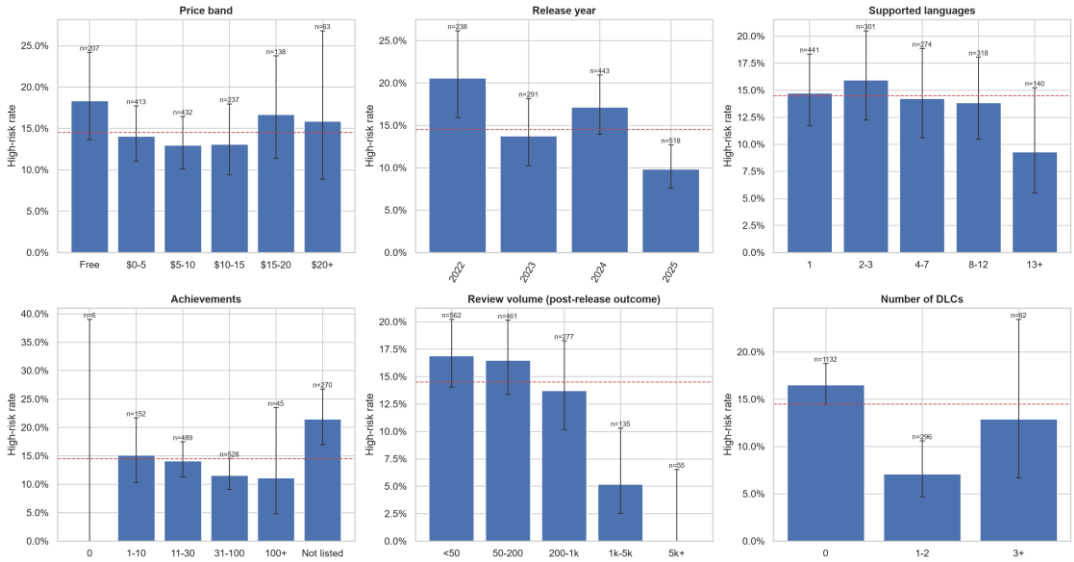


Figure 14. Share of Struggling games by price band, release year, languages, DLC, achievements and review volume with 95% intervals (Python/Seaborn; notebooks/02_EDA_Visualizations.ipynb).

Appendix C. Implementation details and dashboard user guide

Install: `python -m venv .venv`, activate, `pip install -r requirements.txt`. **Features and checks:** `python -m src.features`. **Models:** `python -m src.models.review_models, game_models, review_xgboost, game_xgboost; python -m src.models.compare`. **Tuning:** `python -m src.models.grid_search --task game --model xgboost`. **Report:** `python -m src.report.build_capstone_report`. **Dashboard:** `streamlit run src/dashboard/app.py`. **User guide:** *Market explorer*: filter by release year, price band, minimum reviews and tags; KPI cards and charts update. *Tag rules and complaints*: set stability, minimum lift and a tag or price to filter rules; open the complaint-themes and refund-window tabs. *Patch impact*: read the raw and matched-control results. *Launch-*

risk scorer: describe a planned game (price, launch date, platforms, tags, categories) or load an existing game and press *Score this game*; read the predicted tier and scores (not probabilities).

Appendix D. Tracking-board evidence, contribution records, AI declaration and similarity report

Table 17. Cards per assignee on the project board (68 cards, all in Completed; a card with two assignees counts for both).

Team member	Cards assigned	Review 1	Review 2
Shantharam P	39	20	19
Gayas Sheik	14	5	9
Thotakura Srihitha	11	2	9
Pradeep Sasidharan	6	3	3
Valikala Tejaswini	7	2	5

Project board: https://github.com/users/Shan713/projects/1 (board history and card evidence). Commit and pull-request history: https://github.com/Shan713/IndieGuard/pulls. The signed originality and AI-use declarations are on the declaration page and in Section 8.3. Turnitin similarity report: [attach report; similarity __%].